

SQL

- declarative: what output should look like

Commands

- **SELECT** <columns> ← * to select all
- **FROM** <table(s)>
- **AS** <name>
- **WHERE** <conditions> ← =, !=, >, <, >=, <=, AND, OR
- **ORDER BY** <columns> [ASC/DESC]
- **GROUP BY** <column(s)>
- **HAVING** <conditions>
 - Aggregation functions: COUNT, MIN, MAX, AVG, SUM
 - used in HAVING or SELECT w/ GROUP BY
 - applied after filtering & grouping
- **JOIN... ON...**
 - **SELECT** <cols> **FROM** <table1> **JOIN** <table2> **WHERE** <condition>
 - every possible combo.
 - only for rows we want
 - **SELECT** <cols> **FROM** <table1> **JOIN** <table2> **ON** <condition>
- **LIMIT** <#>

Other Commands

- **CREATE TABLE** <name> **AS** <select statement>
- **INSERT INTO** <table (cols,...)> **VALUES** <vals,...>
- **UPDATE** <table> **SET** <col> = <value> [**WHERE** <condition>];

Tips

- first: consider columns to be displayed, then select/ordering
- avoid duplicates when combining table w/ itself
- specify which table to pull a column from if columns w/ same names

① Houses & Restaurants (CSM FA23)

Houses

Name	Price	Area	Neighborhood	Bedrooms
Shattuck	10	150	Downtown	1
Bancroft	20	150	Southside	4

Restaurants

Name	Neighborhood
Cafe 3	Southside
Imm Trail	Downtown

- order houses table least to greatest by price per bedroom:


```
SELECT * FROM houses ← all columns
ORDER BY price/bedrooms; ← least to greatest
```
- produce a column table w/ NBH that have houses w/ total area of at least 200:


```
SELECT Neighborhood, sum(Area) ← aggregate func to combine into from grouping
FROM houses
GROUP BY Neighborhood ← grp to analyze
HAVING sum(Area) >= 200; ← to only have NBH w/ area > 200
```
- produce table of NBHs w/ both restaurants and houses w/ more than 2 bedrooms:


```
SELECT DISTINCT houses.Neighborhood ← so NBH not listed twice
FROM houses, restaurants
WHERE restaurants.neighborhood = houses.neighborhood
AND houses.bedrooms > 2;
```

② Orders & Menu (Lab 11)

Orders

name	item	ice
Christy	Black Milk Tea	Regular
Jessica	Special Tea	Light

Menu

item	price
Black Milk Tea	5
Wintermelon Milk Tea	6

Boba

name	ice	pieces
Black Milk tea	Regular	23
Wintermelon	Light	22

- returns names of people w/ same order


```
SELECT o1.name, o2.name
FROM orders AS o1, orders AS o2
WHERE o1.item = o2.item AND o1.name < o2.name;
```
- average pieces of boba for each option for ice excluding 'Thanos Milk Tea'. 2 columns (ice | pieces) & 3 rows (per ice option)


```
SELECT ice, AVG(pieces) AS avg-pieces
FROM boba
WHERE name <> 'Thanos Milk Tea'
GROUP BY ice;
```

③ SQL for Sequels

Books

name	number	series
Sorcerer's Stone	1	Harry Potter
Chamber of Secrets	2	Harry Potter
Catching Fire	2	Hunger Games

Series Author

series	author	country	city
Harry Potter	J.K. Rowling	UK	Yate
Hunger Games	Suzanne Collins	USA	Hartford
Divergent	Veronica Roth	USA	New York

- output series w/ > two books in books table


```
SELECT series
FROM books
GROUP BY series
HAVING count(*) > 2;
```
- each book name w/ name of direct sequel & prequel listed before sequel


```
SELECT prequel.name, sequel.name
FROM books AS prequel, books AS sequel
WHERE prequel.series = sequel.series
AND prequel.number + 1 = sequel.number;
```
- output book name, author, author's city for books in books table, second in series w/ authors from USA; alphabetically by author


```
SELECT name, author, city
FROM books, series_authors
WHERE books.series = series_authors.series
AND number = 2
AND country = "USA"
ORDER BY author
```

④ Student & Building (Sp22 Final)

Student

name	grade	res_hall
Sebastian	Freshman	Triumph
Jessica	sophomore	Triumph

Building

res_hall	complex	year
Triumph	Unit A	1980
Empire	Unit A	1976

- name & res_hall of all sophomore students


```
SELECT name, res_hall
FROM student
WHERE grade = 'sophomore'
```
- building rows w/ unit A, alphabetical by name


```
SELECT * FROM building
WHERE complex = 'Unit A'
ORDER BY res_hall
```
- total # junior students who live in each comp.


```
SELECT complex, count(*) AS total
FROM student AS s, building AS b
WHERE s.res_hall = b.res_hall
AND grade = 'Junior'
GROUP BY complex
```

5 Aggregate Functions in HAVING

```
SELECT department_id, Avg (salary) AS avg-salary  
FROM employees  
GROUP BY department_id  
HAVING Avg (salary) > 1000
```

Trees

- hierarchical recursive data structure
- Implementation
 - Instantiating a tree instance: `Tree(value), Tree(value, branches)`
 - `is_leaf`: returns true if tree is a leaf
 - `t.value`: value of root node
 - `t.branches`: list of branches, each a tree

Recursion Syntax

- recursive steps
 - for `b in t.branches`: ← for loop to continue recursion b/c recursive_call(b) every `b in t.branches` = Tree instance
 - `[recursive_call(b) for b in t.branches]` ← useful in i.e. sum()
- base cases
 - if `tree.is_leaf()`: ← true if reached leaf

Mutative Problems

- `def mutate_tree(t):`
 - `<modify t.value &/or t.branches>`
 - `for b in t.branches OR for i in range(len(t.branches)):` ← multiple trees
 - `mutate_tree(b)`
- recursive calls w/in for loop
- if tree a leaf, won't enter for loop

Non-Mutative

- `def return_tree(t):`
 - `new_value = <modified value>`
 - `new_branches = []`
 - `for b in t.branches OR for i in range(len(t.branches)):`
 - `new_branches += [return_tree(b)]`
 - `return Tree(new_value, new_branches)`

returns Tree

- `def misc_tree(t):`
 - `result = <some start value: involve t.value>`
 - `for b in t.branches OR for i in range(len(t.branches)):`
 - `# make recursive call misc_tree(b), never alone`
 - `# update result based output of recursive call`
 - `return result`

do smth else

- if `non_mutative(b) == value: <do smth>`
- `result += non_mutative(b)`
- `result = max(non_mut(b), result)`
- `result.append(non_mut(b))`

① `def mutative_prune(t, val):`

```
""" mutates t s.t subtrees w/ val at
root node removed:
"""
t = Tree(1, [Tree(3, [Tree(2)])])
mutative_prune(t, 3) → Tree(1)

t.branches = [b for b in t.branches if b.value != val]
for b in t.branches:
    mutative_prune(b, val)
```

② `def prune_tree(t, val):`

```
""" non-mutative prune """
new_branches = []
for b in t.branches:
    if b.value != val:
        new_branches.append(prune_tree(b))
return Tree(t.value, new_branches)
```

③ `def count_nodes(t, val):`

```
""" # nodes in t != val """
result = 0
if t.value != val:
    result += 1
for b in t.branches:
    result += count_nodes(b, val)
return result
```

④ SymmetricTree (Sp 20 final)

```
def symmetric_decreasing_tree(n):
    """ nodes w/ 2 branches & values decrease by 1 as depth increases """
    if n == 1:
        return Tree(n)
    else:
        branches = [symmetric_decreasing_tree(n-1), symmetric_decreasing_tree(n-1)]
        return Tree(n, branches)
```

⑤ `def sum_evens(tree):`

```
""" sum even levels of tree """
def helper(t, depth):
    total = 0
    if depth % 2 == 0:
        total += t.value
    return total + sum([helper(b, depth+1) for b in t.branches])
return helper(tree, 0)
```

⑥ `def sequential_prune(t):`

```
""" prunes tree to keep nodes sequential down depth """
t.branches = [sequential_prune(b) for b in t.branches if t.value + 1 == b.value]
return t
```

⑦ Apple Trees

```
def count_apples(t):
    """ good apples = value 1, count good """
    if t.is_leaf() and t.label:
        return 1
    elif t.is_leaf():
        return 0
    else:
        count = 0
        if t.label == 1:
            count += 1
        for b in t.branches:
            count += count_apples(b)
        return count
```

```
def pick_apples(t):
    """ rotten = value 0, remove it & add branches
    to parent node """
    for b in t.branches:
        if b.label == 0:
            t.branches.extend(b.branches)
            t.branches.remove(b)
            return
    for b in t.branches:
        pick_apples(b)
```

⑧ Slicer Tree (Sp 22 final)

```
def find_recommenders(t, star_students):
    """ return list of 2 element tuples w/ star student & recommender """
    result = []
    for b in t.branches:
        if b.value in star_students:
            result += [(b.value, t.value)]
        result += find_recommenders(b, star_students)
    return result
```

⑩ eval_with_add(a+a.14)

def eval_with_add(t):

"evaluate expression tree"

if t.value != '+':

return t.value

res = 0

for b in t.branches:

res += eval_with_add(b)

return res

⑪ def apply_tree(fn_tree, val_tree):

"mutate val_tree by applying functions"

val_tree.value = fn_tree.value(val_tree.value)

for i in range(len(val_tree.branches)):

apply_tree(fn_tree.branches[i], val_tree.branches[i])

⑫ Combine Tree (disc 9)

def combine_tree(t1, t2, combiner):

combined = []

for i in range(len(t1.branches)):

combined.append(combine_tree(t1.branches[i], t2.branches[i], combiner))

return Tree(combiner(t1.value, t2.value), combined)

⑬ Cherry Tree Blossom

def cherry_tree_blossom(t):

"Increment node by parent's value from top"

for b in t.branches:

b.value += t.value

cherry_tree_blossom(b)

OOP/Inheritance

- classes = blueprints for instances of objects
- each class represents a type or category of objects
- classes have attributes & methods

• Cars Example:

- **class** - blueprint of car
- **constructor** - building the car
- **instance of class** - each individual car
- **class attributes** - attributes that all cars share (4 wheels)
- **instance attributes** - differing attributes among cars (paint color)
- **methods** - behaviors all cars have (drive forward)

• class Car: *class name*

dec wheels = 4 *class attribute*

Constructor *→* def __init__(self, color, brand, model):
self.fuel = 10
self.color = color
self.brand = brand
self.model = model

self (instance)
Instance attributes

def repaint(self, new_color): *← method*
self.color = new_color

creates car object w/ init

myCar = Car('green', 'Lambo', 'idk')

myCar.wheels *→* 4

myCar.color *→* green

myCar.repaint('blue') *→ myCar.color → 'blue'*

Car.repaint(myCar, 'red') *" " → 'red'*

• Inheritance

- "is a" relationship
- reduce amount of code repeated
- **class subclass (superclass):**
define methods
- attributes & methods inherited
- methods w/ same name overridden

• super()

- **super()** method to access superclass methods

• class BaseAccount:

```
def __init__(self, name, initial_deposit):  
    self.name = name  
    self._acct_no = Account._account_number_seed  
    Account._account_number_seed += 1  
    self._balance = initial_deposit
```

```
def CheckingAccount(BaseAccount):
```

```
    def __init__(self, name, initial_deposit):  
        BaseAccount.__init__(self, name, initial_deposit)  
        # OR  
        super().__init__(name, initial_deposit)  
        self._type = "checking"
```

• Lookup Rules

- **Instance variable**: lookup name in instance *→* in class that instance belongs *→* parent *→* error
- **Class variable**: lookup in class *→* parent class (recursively) *→* error if nothing
- **Method**: lookup in class *→* lookup in parent class

① Color Schemes (sp22 final q9)

• class Color:

```
def __init__(self, red, green, blue):
```

```
    self.r = red  
    self.g = green  
    self.b = blue
```

```
def dissimilarity(self, other): → computes dissimilarity btwn 2 colors  
    r_diff = abs(self.r - other.r) & dissimilarity = sum of abs diff btwn r, g, b  
    g_diff = abs(self.g - other.g) attributes of 2 colors  
    b_diff = abs(self.b - other.b)  
    return r_diff + g_diff + b_diff
```

• class ColorPalette:

```
def __init__(self, color_list, threshold):
```

```
    self.threshold = threshold  
    self.colors = []  
    for c in color_list:  
        self.add_to_palette(c)
```

```
def add_to_palette(self, color) → don't add a color if less than threshold, else  
    for c in self.colors: print "Too similar!"  
        if color.dissimilarity(c) < self.threshold:  
            print("Too similar!")  
            return  
    self.colors += [color]
```

- **class TriColor(ColorPalette)**: *→ is a palette & computes third color, the avg of 2 input colors, 3 colors form color palette w/ similarity threshold*

```
def __init__(self, c1, c2, threshold):
```

```
    avg_r = (c1.r + c2.r) / 2  
    avg_g = (c1.g + c2.g) / 2  
    avg_b = (c1.b + c2.b) / 2  
    avg_color = Color(avg_r, avg_g, avg_b)
```

```
    super().__init__([c1, c2, avg_color], threshold)
```

② Final Countdown (sp23 q16)

class Question:

Serial_no = 1000

def __init__(self, text, solution, points):

self.text = text

self.solution = solution

self.points = points

self.serial_no = Question.serial_no

Question.serial_no += 1

} for
unique serial num

def grade(self, student_sol):

if student_sol == self.solution

return self.points

else:

return 0

} grade 0's

class Exam:

def __init__(self, questions, submissions):

""" Exam([q1, q2], [{1000: "False", 1001: "True"}, {...}]) """

{from Questions}

self.questions = questions

self.submissions = submissions

self.scores = []

def total_score(self):

return sum([q.points for q in self.questions])

} total pts on exam,
sum all question points

def grade_submission(self, submission):

← {1000: "ans", ...}

score = 0

for q in self.questions

score += q.grade(submission[q.serial_no])

return score

} score of student

def grade_all(self):

for s in self.submissions:

self.scores.append(self.grade_submission(s))

} add overall scores to list

- Question ≠ subclass of Exam b/c exam has methods that questions shouldn't have ("is a")

Iterators / Generators

• Iterable

- object that can be iterated through
- can do: `for _ in iterable_name`
- Ex: lists, dicts, strings, range, tuples

• Iterator

- object that saves position in an iterable
- process iterable sequentially
- bookmark (iterator) saves position in a book (iterable)
- Methods
 - `iter(<iterable>)`: pass in iterable, returns iterator
 - `next(<iterator>)`: moves iterator to next position on iterable (StopIterationError)

• Generator

- type of Iterator
- instead of class, it's a function that produces sequence of results
- Yield instead of return
 - list on it → everything after yield
 - → what's yielded before is gone
- resume after yield
- uses while or for
 - while True for infinite generators

• Iterator:

• class evensTillN:

```
def __init__(self, n):
    self.curr = 0
    self.final = n
def __iter__(self):
    return self
def __next__(self):
    if self.curr < self.final:
        temp = self.curr
        self.curr += 2
        return temp
    else:
        raise StopIteration
```

• Generator:

• def evens_till_n(n):

```
i = 0
while i < n:
    yield i
    i += 2
```

• def evens_infinite(n):

```
i = 0
while True:
    yield i
    i += 2
```

① Alternating Generator (Sp21 final)

```
def alternating_generator(gen1, gen2, gen3):
```

```
    """ cycles through yielding from each generator """
```

```
    iter_list = [gen1, gen2, gen3]
```

```
    while True:
```

```
        for i in iter_list:
            yield next(i)
```

② Maze (Sp22 final)

```
def check(correct_gen, guess_gen):
```

```
    """ 2 iterators w/ correct & guessing path. A
    game to lose or complete """
```

```
    while True:
```

```
        try:
```

```
            correct_step = next(correct_gen)
            guess_step = next(guess_gen)
            if correct_step != guess_step:
                yield 'Lose'
            return
```

```
        except:
            yield 'OK'
```

```
    except StopIteration:
```

```
        yield 'Maze Complete'
```

③ One Giant Leap Forward (fa20 final)

```
""" l1 = Link(1, Link(2, Link(3, Link(4, Link(5)))))
```

```
↳ [1, 3, 2, 4, 3, 5]
```

```
class LinkedListIterator:
```

```
    def __init__(self, lnk):
```

```
        self.pointer1 = lnk
```

```
        self.pointer2 = lnk.rest.rest
```

```
        self.counter = 0
```

```
    def __iter__(self):
```

```
        return self
```

```
    def __next__(self):
```

```
        if self.pointer2 is Link.empty:
```

```
            raise StopIteration
```

```
        if self.counter % 2 == 0:
```

```
            return_value = self.pointer1.first
```

```
            self.pointer1 = self.pointer1.rest
```

```
        else:
```

```
            return_value = self.pointer2.first
```

```
            self.pointer2 = self.pointer2.rest
```

```
        self.counter += 1
```

```
        return return_value
```

④ def three_flips(coin):

```
    """ iterator coin w/ H/T & yields sequence
    of triples of coin flip outcomes """
```

```
    start = next(coin) + next(coin) + next(coin)
```

```
    while True:
```

```
        yield start
```

```
        start = start[1:] + next(coin)
```

⑤ def distinct_G(i):

```
    """ return generator of distinct elements
    in iterable i """
```

```
    items = []
```

```
    for elem in i:
```

```
        if elem not in items:
```

```
            items += [elem]
```

```
            yield elem
```

⑥ Tree Farm (fa21 final)

```
def tree_mismatches(t1, t2):
```

```
    if t1.value != t2.value:
```

```
        yield t1.value, t2.value
```

```
    n = len(t1.branches)
```

```
    for i in range(n):
```

```
        branch_mismatches = tree_mismatches(t1.branches[i], t2.branches[i])
```

```
        for val in branch_mismatches:
```

```
            yield val
```

* 2 trees, nodes in same position
but w/ diff values as tuple

⑦ Tree Sort (sp23 final)

```
def tree_sort(t): ← yields all values from tree in ascending order
```

```
    sorted_branches = [tree_sort(b) for b in t.branches] → returns generators
```

```
    next_smallest = [next(gen) for gen in sorted_branches] → goes thru gens & yields smallest
```

```
    value_yielded = False → indicator var
```

```
    while len(next_smallest) > 0:
```

```
        min_index = min(range(len(next_smallest)), key = lambda i: next_smallest[i])
```

```
        if not value_yielded and t.value < next_smallest[min_index]
```

```
            yield t.value
```

```
            value_yielded = True
```

```
        yield next_smallest[min_index]
```

```
    try:
```

```
        next_smallest[min_index] = next(sorted_branches[min_index])
```

```
    except StopIteration:
```

```
        next_smallest.pop(min_index)
```

```
        sorted_branches.pop(min_index)
```

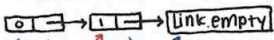
```
    if not value_yielded:
```

```
        yield t.value
```

Linked Lists

- represent sequence of ordered values
- Each link has first & rest attribute
 - first: value contained at the link
 - rest: remainder of linked list - link instance or `Link.empty`
- recursive data structure
- Construction

• `lst = Link(0):` 

• `lst = Link(0, Link(1)):` 

» `lst.first` → 0

» `lst.rest.first` → 1

» `lst.rest.rest` is `Link.empty` → True

• `lnk = Link(2, Link(3, Link(4))):` 

» `lnk.rest = lnk.rest.rest`

To remove node 3

Common motifs

- Iterative Pattern
 - while `l` is not `Link.empty`:
 - [Do stuff]
 - `l = l.rest`

Recursive Pattern

- if `l` is `Link.empty`:
 - [BASE CASE]
- else:
 - [RECURSIVE STEP ON `l.rest`]

Linked List & Recursion

- Base case:
 - End of linked list? → `lnk` is `Link.empty`
 - special conditions?
- Recursive case:
 - keep looking through rest of linked list
 - recursively call on rest of linked list: `func(..., lnk.rest, ...)`
- Mutative
 - remember our attributes first & rest
 - to change stored value/link, reassign attributes
- Non-Mutative
 - create new linked list using `Link(...)` constructor to be returned
 - typically uses values from existing linked list, obtained w/ attributes

① Remove (non-mutative)

```
def remove(lnk, n):
    """ >>> lnk = Link(1, Link(3, Link(3)))
    >>> remove(lnk, 3) → Link(1)
    >>> lnk → Link(1, Link(3, Link(3))) """
    if lnk is Link.empty:
        return Link.empty
    elif lnk.first == n:
        return Link(remove(lnk.rest, n))
    else:
        return Link(lnk.first, remove(lnk.rest, n))
```

- Base case
 - when do we stop?
 - Reaching `Link.empty`
- Recursive case
 - Conditions to check? Compare w/ `n`
 - What case not handled yet? `first != n`
 - Do we need to keep looking? Yes
- Return
 - Mutative or nonmutative?
 - non-mutative
 - return `Link(...)`

② WordBank (non-mutative)

```
def wordBank(lnk, lst):
    """ keep lnk word if ≤ lst[0] or length """
    if lnk is Link.empty:
        return Link.empty
    elif len(lnk.first) ≤ lst[0]:
        return Link(lnk.first, wordBank(lnk.rest, lst[1]))
    else:
        return Link(wordBank(lnk.rest, lst[1]))
```

- Base case:
 - when do we stop? `Link.empty`
- Recursive case:
 - special condition to check?
 - `len(...) ≤ lst[0]`
 - case not handled yet?
 - `len(...) > lst[0]`
 - keep looking? yes
- Return? Non-mutative & `Link(...)`

③ Merge (non-mutative)

```
def merge(lnk1, lnk2):
    """ lnk1 of strings, if longer than lnk2 of
    ints, merge string first """
    if lnk1 is Link.empty:
        return Link.empty
    elif len(lnk1.first) > lnk2.first:
        return Link(lnk1.first, Link(lnk2.first, merge(lnk1.rest, lnk2.rest)))
    else:
        return Link(lnk2.first, Link(lnk1.first, merge(lnk1.rest, lnk2.rest)))
```

④ Remove-Multiple (Fa 21 Final)

```
def remove_multiple(lnk, number_to_remove):
    """ w/ [2,7], remove next 2, keep node, remove next 7... """
    if lnk is Link.empty or lnk.rest is Link.empty or not number_to_remove:
        return
    else:
        for i in range(number_to_remove[0]):
            if lnk.rest is not Link.empty:
                lnk.rest = lnk.rest.rest
            else:
                return
        remove_multiple(lnk.rest, number_to_remove[1:])
```

```
⑤ def contains(lnk, n):
    if lnk is Link.empty:
        return False
    elif lnk.first == n:
        return True
    else:
        return contains(lnk.rest, n)
```

```
⑥ Replace non-mutative
def replace(lnk, n):
    """ replace n w/ 0 """
    if lnk is Link.empty:
        return Link.empty
    elif lnk.first == n:
        return Link(0, replace(lnk.rest, n))
    else:
        return Link(lnk.first, replace(lnk.rest, n))
```

```
⑦ def separate(num, lnk):
    """ Separates a consecutive w/ num """
    if lnk is Link.empty:
        return Link.empty
    elif lnk.rest is Link.empty:
        return Link(lnk.first)
    elif lnk.first == lnk.rest.first:
        return Link(lnk.first, Link(num, separate(num, lnk.rest)))
    return Link(lnk.first, separate(num, lnk.rest))
```

⑧ out of order

def out_of_order(link):

if link is link.empty or link.rest is link.empty:

← return number out of order & mutate list to skip over & None if sorted

return None

if link.first > link.rest.first

unsorted_value = link.rest.first

← value makes it unsorted

link.rest = link.rest.rest

return unsorted_value

return out_of_order(link.rest) ← sorted]

Exceptions

- errors that halt execution of program
- raise exception when it occurs
- Exception Types
 - AssertionError - fail assert statement
 - TypeError - wrong num/type in argument
 - NameError - name not found
 - KeyError - key not in dictionary
 - RuntimeError - catch-all troubles
- Handling Exceptions
 - try:
 <try suite>
except <exception class> as <name>: # <name> optional
 <except suite>
 # multiple except blocks
- Assertion statements
 - assert <condition>, <error message>
 - If condition false → raise AssertionError
 - otherwise, do nothing
- Raising Exception
 - raise <exception-type-or-object>
- Exception Class
 - class CustomError(Exception):
 pass
 - class FancyError(Exception):
 def __init__(self, message, extra_data):
 super().__init__(message)
 self.extra_data = extra_data

① Concept Man (Sp 23 Final)

d = {1: 2} ←

```
try:  
    print(d[1]) # d[2] # d[1/0] → 2  
except NameError as e: → donkey  
    print('Nyaaaa') → wowzas!  
except KeyError as e:  
    print('donkey') ←  
except ZeroDivisionError as e:  
    print('wowzas!') ←
```

Efficiency

• Big O notation

- how number of operations change as input size changes
- **asymptotic behavior** - efficiency when n gets big

• Big O examples

• $O(1)$

- constant growth - doesn't affect time
- retrieving element from a list
- adding element to front of linked list
- appending to a list
- setting/retrieving key-value pair in dict
- Basic operations (arithmetic, bool operators, variable assignment)

• $O(\log n)$

- logarithmic growth - increments time by a constant
- going down height of a (balanced) tree
- binary search

• $O(n)$

- linear growth - increases time by a constant
- linear search
- inserting into a random location in a list
- recursive factorial
- iterating over a list of length n

• $O(2^n)$

- exponential growth - multiplied time by a constant
- recursive fibonacci

• $O(n^2)$

- quadratic growth - increases time by n times a constant
- finding all pairs of a list of integers
- double for loop

• Big O Tips

- "worst case scenario" asymptotic behavior
 - drop lower order terms: $O(n^2 + n) = O(n^2)$
 - drop multiplicative constants: $O(3n^2) = O(n^2)$
- If you see a loop
 - determine runtime of code inside loop
 - multiply runtime of 1 loop by number of loops
- If recursion
 - draw tree diagram
 - tree recursion often exponential
- try different inputs & see how operations change

① JoJo's Bizarre Squared-Venture (sp23 q12)

```
def squared_platinum(n):
    res = 0
    for za-warudo in range(n):
        res += n
    return res
O(n)
```

② def squared_mit_purple(n):

```
    return n * n
O(1)
```

③ def squared_experience(n):

```
    if n % 2 == 0:
        return 4 * square(n // 2)
    return n * n
O(log(n))
```

```
④ def squared_q_diamond(n):
    res = 0
    for kira-queen in range(n):
        for j in range(m):
            res += 1
    return res
O(n^2)
```

⑤ Efficiency Examples (Review Section)

```
x = x + 5 # O(1)
y = 1000 + 2 # O(1)
print(x + y) # O(1)
```

$3 \times O(1) \rightarrow O(1)$

```
for i in range(0, n): # runs n times
    print(i) # O(1)
```

$n \times O(1) \rightarrow O(n)$

```
a = 10 ** 2 # O(1)
for i in range(0, n/2): # runs n/2 times
    print(i) # O(1)
```

$O(1) + (n/2) \times O(1)$
 $= O(1 + n/2)$ (drop lower order)
 $= O(n/2)$ & constants
 $\rightarrow O(n)$

```
if n % 3 == 1: # O(1)
else: # O(n log n)
```

$O(1)$ or $O(n \log n)$
 $\rightarrow O(n \log n)$ worst case

⑥ def fclist: # len(lst) = n

```
    result = []
    for elem in lst: # runs n times
        if elem % 2 == 0: # O(1)
            result.append(elem)
```

$n \times O(1) \rightarrow O(n)$

return result

⑦ def q(n):

```
    i = n
    while i >= 1:
        print(i)
        i /= 2 # binary
```

$\rightarrow O(\log n)$

⑧ def h(lst):

```
    pairs = []
    for i in range(len(lst)):
        for j in range(i, len(lst)):
            pairs.append([i, j])
```

double for loop
 $\rightarrow O(n^2)$

return pairs